STAT 6206 Tutorial

Department of Statistics and Data Science

March 12, 2026

香港中文大學
The Chinese University of Hong Kong

Overview

- Motivating Example
- Bayesian Software
- Estimating with PyMC
- Two Real Examples



Section 1

Motivating Example



香港中文大學
The Chinese University of Hong Kong



Motivating Example

Poisson GLM Model

A hospital chain investigates factors influencing monthly infection counts(y) across 100 branches.

Suppose the true model to generate y is:

$$y \sim \text{Poisson}(\mu)$$

$$log(\mu) = \beta_0 + \beta_1 x_1 + \beta_2 x_2,$$

- x_1: Standardized hospital size (beds)
- x_2: Handwashing compliance rate
- $\beta = (\beta_0, \beta_1, \beta_2) = (0.5, 1, -1)^T$

Key Questions: How to recover true β parameters from simulated data?



Section 2

Bayesian Software



香港中文大學
The Chinese University of Hong Kong

BUGS:

BUGS, short for **Bayesian inference Using Gibbs Sampling**, is one of the earliest widely used Bayesian modeling languages. The official BUGS project page now states that BUGS/WinBUGS/OpenBUGS are **no longer actively developed.**

Adventages:

1. It is **historically important** and very well documented in textbooks and older literature.
2. It is good for introducing **hierarchical Bayesian thinking**, especially the prior–likelihood–posterior structure.

Disadventages:

1. The **software stack is old** and It is often **less efficient** for high-dimensional continuous models with strong posterior correlations

JAGS:

JAGS stands for **Just Another Gibbs Sampler**. It was designed to be extensible so that users can write their own functions, distributions, and samplers.

Adventages:

1. **Cross-platform support**, which makes it more convenient than the older WinBUGS workflow.
2. Well suited for traditional Bayesian hierarchical models

Disadventages:

1. It is often less efficient than Stan for complex continuous-parameter models.
2. Mixing and diagnostics can be less favorable in high-dimensional correlated posteriors

Stan:

Stan is an open-source project that officially started in 2011. Its major distinction from BUGS/JAGS is that Stan emphasizes **gradient-based computation**, especially HMC and NUTS, rather than classical Gibbs sampling.

Adventages:

1. Its gradient-based inference is much more efficient for complex continuous-parameter models
2. It has a mature diagnostic framework, which makes it particularly useful for teaching MCMC diagnostics such as R-hat, ESS, and divergences.

Disadventages:

1. Stan cannot directly sample discrete latent parameters

2. It has a steeper learning curve for beginners, because users need to understand Stan syntax, compilation, and the logic of HMC-based inference

PyMC:

PyMC is a Bayesian/probabilistic programming framework in Python, which brought in automatic differentiation and modern gradient-based inference such as HMC/NUTS.

Adventages:

1. It integrates naturally with the Python data-science ecosystem, including NumPy, pandas
2. It is highly flexible for custom models, which is useful for research prototyping and non-standard likelihoods.

Disadventages:

1. Its flexibility means the user often has to manage more implementation details
2. For very standard GLMs, it is not always as concise as specialized high-level interfaces



Section 3

Estimating with PyMC

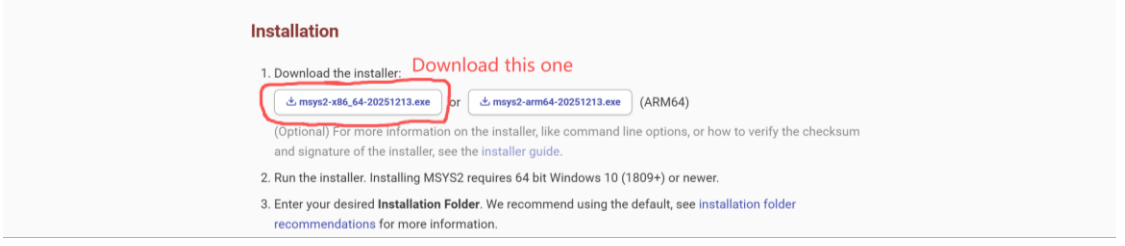


香港中文大學
The Chinese University of Hong Kong

Download MSYS2 (for windows):

- Why: Because PyTensor, which underlies PyMC, looks for g++ to compile some low-level C/C++ code; if g++ is unavailable, PyTensor falls back to pure Python, and the code can still run but may be much slower. **On macOS, you usually do not need MSYS2.**

- MSYS2 can be downloaded from the following website: https://www.msys2.org/

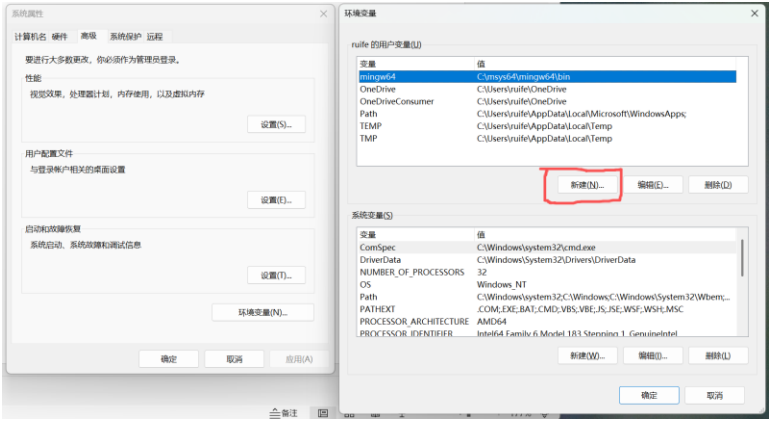


Add the compiler directory to the PATH:

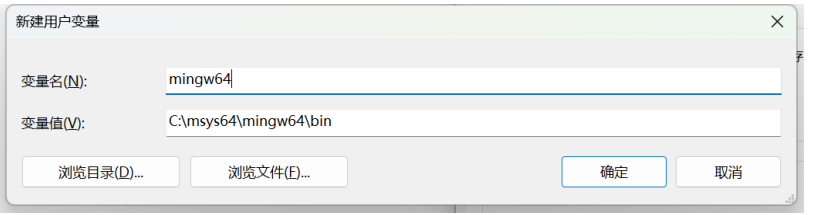
1. Find environment variable



2. Click “add”



3. Add the compiler directory to the PATH



You can also download it to different disk and add corresponding path



Estimating with “PyMC”

Download MSYS2

Add your path in the beginning and import necessary package

```
import os
from multiprocessing import freeze_support

# The following sentence must be putted before import pymc
os.environ["PATH"] = r"C:\msys64\mingw64\bin;" + os.environ.get("PATH", "")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pymc as pm
import arviz as az
```



Estimating with "PyMC"

Generate Data

- Specify random seed, sample size, true value parameter
- Generate x1 and x2, Calculate linear predictor and y, construct a data frame

```
def main():
    rng = np.random.default_rng(666) # random seed
    n = 500 # sample size
    beta_true = np.array([0.5, 1.0, -1.0]) # true value of parameter

    x1 = rng.normal(0, 1, n) # generate x1
    x2 = rng.uniform(0, 1, n) # generate x2

    linear_predictor = beta_true[0] + beta_true[1] * x1 + beta_true[2] * x2
    mu = np.exp(linear_predictor)
    y = rng.poisson(lam=mu) # Calculate linear predictor and y

    # Construct data as a data frame
    df = pd.DataFrame({"y": y,"x1": x1,"x2": x2})
```



Estimating with "PyMC"

Write PyMC code

- Then, we need to specify the model code in PyMC

```
with pm.Model() as pois_glm:
    x1_data = pm.Data("x1_data", df["x1"].values)
    x2_data = pm.Data("x2_data", df["x2"].values)

    # Set prior
    beta0 = pm.Normal("beta0", mu=0, sigma=10)
    beta1 = pm.Normal("beta1", mu=0, sigma=5)
    beta2 = pm.Normal("beta2", mu=0, sigma=5)

    # Linear predictor and link function
    eta = beta0 + beta1 * x1_data + beta2 * x2_data
    mu_model = pm.Deterministic("mu", pm.math.exp(eta))

    y_obs = pm.Poisson("y_obs", mu=mu_model, observed=df["y"].values)
```



Estimating with "PyMC"

Write PyMC code

- Then, we set some parameters of MCMC algorithm

```
idata = pm.sample(
   draws=2000,
   tune=2000,
   chains=4,
   cores=4,
   target_accept=0.90,
   random_seed=666,
   return_inferencedata=True
)

ppc = pm.sample_posterior_predictive(idata, random_seed=666)
idata.extend(ppc)
```



Estimating with "PyMC"

Model Diagnostics

- Furthermore, we can view the summary of the model

```
"""
   View the summary of model
"""
summary = az.summary(
   idata,
   var_names=["beta0", "beta1", "beta2"],
   hdi_prob=0.95,
   round_to=3
)
print(summary)

print("\nTrue values:")
print({"beta0": 0.5, "beta1": 1.0, "beta2": -1.0})
```



Estimating with “PyMC”

Model Diagnostics

- We expect to see following table based on before mentioned code
- We can evaluate the goodness of fitting according to the mean, standard deviation, 95% interval, etc.

```
Sampling ... ---------------------------------------- 100% 0:00:00 / 0:00:00
        mean     sd  hdi_2.5%  hdi_97.5%  ...  mcse_sd  ess_bulk  ess_tail  r_hat
beta0  0.459  0.076     0.310      0.609  ...    0.001  2906.656  3348.950  1.001
beta1  1.030  0.036     0.958      1.102  ...    0.000  4026.308  3627.884  1.000
beta2 -0.982  0.126    -1.226     -0.737  ...    0.002  3142.473  3637.317  1.000

[3 rows x 9 columns]

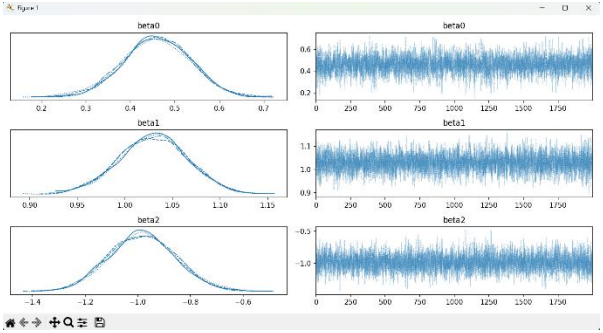
True values:
{'beta0': 0.5, 'beta1': 1.0, 'beta2': -1.0}
```

Estimating with “PyMC”

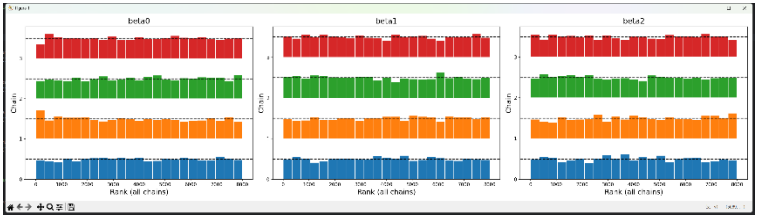
Model Diagnostics

- We expect to see following four plots

Trace plot



Rank plot

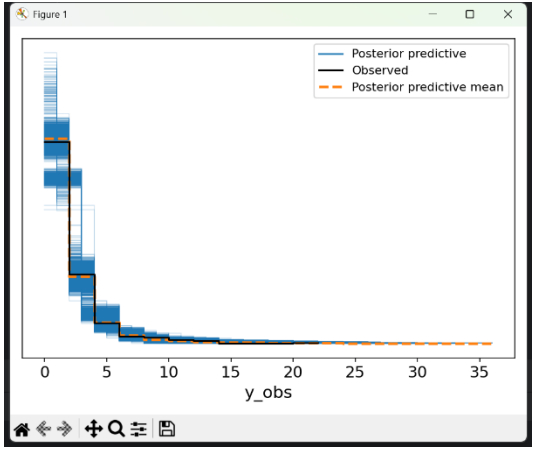


Estimating with “PyMC”

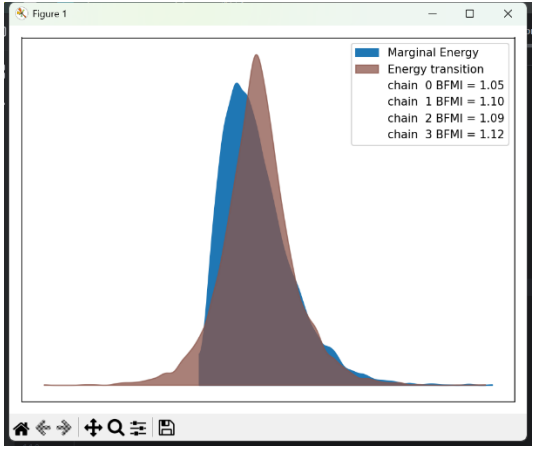
Model Diagnostics

- We expect to see following four plots

PPC plot



Energy plot





Estimating with "PyMC"

Model Diagnostics

- We can also do other model diagnostics

```
print("\nR-hat")
print(az.rhat(idata, var_names=["beta0", "beta1", "beta2"]))

print("\nESS")
print(az.ess(idata, var_names=["beta0", "beta1", "beta2"]))

print("\nBFMI")
print(az.bfmi(idata))

n_div = int(idata.sample_stats["diverging"].sum().values)
print(f"\nNumber of divergences = {n_div}")
```



Estimating with "PyMC"

Model Diagnostics

- Index Interpretation

- **R-hat:** Compare the fluctuation in the chain and difference among chains, If R-hat is close to 1, that means the chain converge
- **ESS:** which means Effective Sample Size, The larger the ESS, the more stable the posterior estimate
- **BFMI:** Indicates whether the sampler thoroughly explored the energy level, If BFMI is low, the geometry structure of posterior may not good
- **Number of divergence:** Are there any NUTS regions where sampling has failed or is unreliable, The ideal value is 0



Estimating with “PyMC”

Model Diagnostics

- We can also draw some plots to evaluate the goodness of estimate

```
az.plot_trace(idata, var_names=[“beta0”, “beta1”, “beta2”]) # Trace Plot
plt.tight_layout()
plt.show()

az.plot_rank(idata, var_names=["beta0", "beta1", "beta2"]) # Rank plot
plt.tight_layout()
plt.show()

az.plot_ppc(idata, var_names=["y_obs"]) # PPC plot
plt.tight_layout()
plt.show()

az.plot_energy(idata) # Energy plot
plt.tight_layout()
plt.show()
```



Estimating with “PyMC”

Model Diagnostics

- Plot Interpretation

- **Trace Plot:** shows whether the chains mix well and stabilize in the same region.
- **Rank plot:** checks whether different chains are sampling from the same posterior distribution.
- **PPC plot:** checks whether the model can generate data similar to the observed data.
- **Energy plot:** diagnoses whether HMC/NUTS is exploring the posterior geometry efficiently.



Section 4

Two Real Examples



香港中文大學
The Chinese University of Hong Kong

Example 1: Headache data

This data set is obtained from a two-treatment, multiple-period crossover trial in which the number of headaches per week was repeatedly measured for 27 patients. In the first period, each patient received the placebo. In the remaining periods, the patients received a placebo (P) or aspartame (A) in random order, using the double-blind crossover treatment design.

Let

- y_{it}: # of headaches for patient i in period t
- d_{it}: # of days for patient i in period t
- x_{it}: treatment received by patient i in period t.
- 0 = placebo, 1 = aspartame

Two models are considered:

M_1: Poisson GLMM

$$log\,(\mu_{it}/d_{it}) = \beta_0 + \beta_1 x_{it} + u_i, \quad or \quad \mu_{it} = d_{it}\, exp(\beta_0 + \beta_1 x_{it} + u_i)$$

$Where\ y_{it} \mid u_i \sim Poisson(\mu_{it})\ and\ u_i \sim N(0, \sigma^2)$.

M_2: Poisson GLM

$$log\,(\mu_{it}/d_{it}) = \beta_0 + \beta_1 x_{it}, \quad \text{or} \quad \mu_{it} = d_{it}\, exp(\beta_0 + \beta_1 x_{it})$$



Two Real Examples

Example 1: Headache data

Step 1: Read and reshape the dataset

```
df = pd.read_csv("headache.txt",sep=r"\s+",header=None,names=["d", "y", "x",
"NT_raw"],na_values=["NA"])
N = 27,Tmax = 5

x_mat = df["x"].to_numpy().reshape(N, Tmax) # Reshape Data
d_mat = df["d"].to_numpy().reshape(N, Tmax)
y_mat = df["y"].to_numpy().reshape(N, Tmax)

NT = np.sum(~np.isnan(y_mat), axis=1) # Calculate NT
mask = ~np.isnan(y_mat) & ~np.isnan(d_mat)
subj_idx, time_idx = np.where(mask)

x_obs = x_mat[mask].astype(float)
d_obs = d_mat[mask].astype(float)
y_obs = y_mat[mask].astype(int)
```



Two Real Examples

Example 1: Headache data

Step 2: Establish GLMM model

```
with pm.Model() as glmm_model:
    x_data = pm.Data("x_data", x_obs)
    d_data = pm.Data("d_data", d_obs)
    subj_data = pm.Data("subj_data", subj_idx)  # Prepare data

    beta0 = pm.Normal("beta0", mu=0, sigma=5)  # fixed effects and prior
    beta1 = pm.Normal("beta1", mu=0, sigma=5)

    sigma_u = pm.HalfNormal("sigma_u", sigma=2)  # random intercept
    u = pm.Normal("u", mu=0, sigma=sigma_u, shape=N)

    eta = beta0 + beta1 * x_data + u[subj_data] + pm.math.log(d_data)
    mu = pm.Deterministic("mu", pm.math.exp(eta))  # linear predictor with offset

    y_like = pm.Poisson("y_like", mu=mu, observed=y_obs)  # likelihood
```



Step 3: Set parameters of MCMC

```
idata_glmm = pm.sample(
    draws=2000,
    tune=2000,
    chains=4,
    cores=1,
    target_accept=0.9,
    random_seed=666,
    return_inferencedata=True
)

pm.compute_log_likelihood(idata_glmm, model=glmm_model)
ppc = pm.sample_posterior_predictive(idata_glmm, random_seed=666)
idata_glmm.extend(ppc)
```



Step 4: View the summary of the model

```
print(
   az.summary(
      idata,
      var_names=["beta0", "beta1", "sigma_u"],
      hdi_prob=0.95,
      round_to=3))
```



Step 5: Model diagnostics

```
az.plot_trace(idata, var_names=["beta0", "beta1", "sigma_u"])
plt.tight_layout()
plt.show()

az.plot_rank(idata, var_names=["beta0", "beta1", "sigma_u"])
plt.tight_layout()
plt.show()

az.plot_ppc(idata, var_names=["y_like"])
plt.tight_layout()
plt.show()

az.plot_energy(idata)
plt.tight_layout()
plt.show()
```



Step 5: Model diagnostics

```
print("\nR-hat")
print(az.rhat(idata_glmm, var_names=["beta0", "beta1", "sigma_u"]))

print("\nESS")
print(az.ess(idata_glmm, var_names=["beta0", "beta1", "sigma_u"]))

print("\nBFMI")
print(az.bfmi(idata_glmm))

n_div = int(idata_glmm.sample_stats["diverging"].sum().values)
print(f"\nNumber of divergences = {n_div}")
```

Step 6: Establish and run GLM model

```
with pm.Model() as glm_model:
    # Prepare data
    x_data = pm.Data("x_data", x_obs)
    d_data = pm.Data("d_data", d_obs)

    beta0 = pm.Normal("beta0", mu=0, sigma=5) # fixed effects and prior
    beta1 = pm.Normal("beta1", mu=0, sigma=5)

    eta = beta0 + beta1 * x_data + pm.math.log(d_data) # linear predictor with offset
    mu = pm.Deterministic("mu", pm.math.exp(eta))

    y_like = pm.Poisson("y_like", mu=mu, observed=y_obs) # likelihood
```

Step 7: Model Comparison

- **Loo:** Estimates how well a model is expected to predict new data using leave-one-out cross-validation.
- **DIC:** A Bayesian information criterion that balances model fit and effective complexity, with smaller values generally indicating a better model.
- **Waic:** Measures out-of-sample predictive performance while adjusting for model complexity using the full posterior distribution.
- **Rank:** Gives the relative ordering of the candidate models, rank = 0 indicates the best model
- **Warning:** indicates that the corresponding LOO or WAIC estimate may be unreliable and should be interpreted with caution.

Step 7: Model Comparison

Calculate DIC of GLM model

```
def dic_glm(idata, x_obs, d_obs, y_obs):
    beta0_bar = idata.posterior["beta0"].mean().item()
    beta1_bar = idata.posterior["beta1"].mean().item()

    mu_bar = np.exp(beta0_bar + beta1_bar * x_obs + np.log(d_obs))

    ll = idata.log_likelihood["y_like"]
    obs_dims = [d for d in ll.dims if d not in ("chain", "draw")]
    D_bar = (-2 * ll.sum(dim=obs_dims)).mean().item()
    D_hat = poisson_deviance(y_obs, mu_bar)

    return 2 * D_bar - D_hat
```

Step 7: Model Comparison

Calculate DIC of GLMM model

```
def dic_glmm(idata, x_obs, d_obs, y_obs, subj_idx):
    beta0_bar = idata.posterior["beta0"].mean().item()
    beta1_bar = idata.posterior["beta1"].mean().item()
    u_bar = idata.posterior["u"].mean(dim=("chain", "draw")).values

    mu_bar = np.exp(beta0_bar + beta1_bar * x_obs + u_bar[subj_idx] +
np.log(d_obs))

    ll = idata.log_likelihood["y_like"]
    obs_dims = [d for d in ll.dims if d not in ("chain", "draw")]
    D_bar = (-2 * ll.sum(dim=obs_dims)).mean().item()
    D_hat = poisson_deviance(y_obs, mu_bar)

    return 2 * D_bar - D_hat
```

Step 7: Model Comparison

Compare Loo,Waic,DIC of the model

```
cmp_loo = az.compare(
   {"GLMM": idata_glmm, "GLM": idata_glm},ic="loo",method="stacking")
print(cmp_loo)

cmp_waic = az.compare(
{"GLMM": idata_glmm, "GLM": idata_glm},ic="waic",method="stacking")
print(cmp_waic)

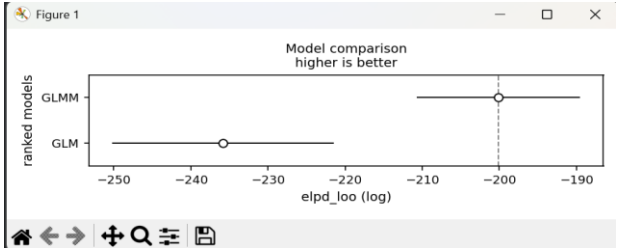
az.plot_compare(cmp_loo)
plt.tight_layout()
plt.show()

dic_glmm = dic_glmm(idata_glmm, x_obs, d_obs, y_obs, subj_idx)
dic_glm = dic_glm(idata_glm, x_obs, d_obs, y_obs)

print(f"GLMM DIC ≈ {dic_glmm:.3f}")
print(f"GLM  DIC ≈ {dic_glm:.3f}")
```

Step 7: Model Comparison

Plot and table interpretation



	rank	elpd_loo	p_loo	...	dse	warning	scale
GLMM	0	-200.126516	24.435764	...	0.000000	True	log
GLM	1	-235.838726	4.094996	...	11.779446	False	log
	rank	elpd_waic	p_waic	...	dse	warning	scale
GLMM	0	-198.710471	23.019719	...	0.000000	True	log
GLM	1	-235.834131	4.090402	...	11.697995	False	log

1. LOO comparison shows that the GLMM outperforms the GLM, as its **elpd_loo** is substantially higher (−200.13 vs −235.84). WAIC leads to the same conclusion (**elpd_waic**: −198.71 vs −235.83) , so both criteria consistently favor the GLMM.

2. The larger **p_loo/p_waic** for the GLMM suggests that it is more flexible and better captures subject-specific heterogeneity

3. However, **warning=True** result indicates that these information-criterion estimates should be interpreted with some caution.

Example 2: Child's Wheeze Data

This example is based on a well-known data set from the Six Cities study, a longitudinalstudy of the health effects of air pollution. 537 children from Ohio, at ages 7, 8, 9, and 10 years were investigated (Chib and Greenberg, 1998). Variables include:

- wheezing status (1 = yes,0 = no) → y_{it}
- mother's baseline smoking status (1 = yes, 0 = no) → S
- children's age → a_{it}

Model Establishment

Let y_{it} denote the wheeze status (1 = yes,0 = no) for child i at age t, and $\eta_{it} = E(y_{it}|u_i) = p_{it}$, A GLMM with the following linear predictor is considered:

$$\eta_{it} = \beta_0 + \beta_1 s_i + \beta_2 a_{it} + u_i + v_{it}$$

- s_i: mother's smoking habit at baseline
- a_{it}: child i at age t
- u_i and v_{it}: independent random effects with distributions $N(0, \sigma_u^2)$ and $N(0, \sigma_{vt}^2)$

Model Establishment

Using the logit link and the linear predictor η_{it}, the GLMM can be rewritten as :

$$logit(p_{it}) = x_{it}^T\beta + z_{it}^T u_i$$

- $x_{it} = (1, s_i, a_{it})^T, \quad \beta = (\beta_0, \beta_1, \beta_2)^T$
- $u_i = (u_i, v_{i1}, v_{i2}, v_{i3}, v_{i4})^T$
- $z_{i1} = (1,1,0,0,0)^T, z_{i2} = (1,0,1,0,0)^T$
- $z_{i3} = (1,0,0,1,0)^T, z_{i4} = (1,0,0,0,1)^T$
- $u_i \sim N(0, \sum)$, with $\sum = (\sigma_u^2, \sigma_{v1}^2, \sigma_{v2}^2, \sigma_{v3}^2, \sigma_{v4}^2)$



Two Real Examples

Example 2: Child's Wheeze Data

Read Data

```
df = pd.read_csv("ohio.csv")
df = df.drop(columns=["rownames"])

child_codes, child_uniques = pd.factorize(df["id"]) # Child index
n_child = len(child_uniques)

age_order = [-2, -1, 0, 1]
visit_map = {a: i for i, a in enumerate(age_order)} # visit index: age -2,-1,0,1 -> 0,1,2,3
visit_idx = df["age"].map(visit_map).to_numpy()

y = df["resp"].to_numpy() # observed variables
smoke = df["smoke"].to_numpy()
age = df["age"].to_numpy()

coords = {"obs": np.arange(len(df)),
   "child": np.arange(n_child),
   "visit": np.arange(4),}
```



Two Real Examples

Example 2: Child's Wheeze Data

Establish GLMM Model

```
with pm.Model(coords=coords) as glmm_model:
    smoke_data = pm.Data("smoke_data", smoke, dims="obs") # prepare data
    age_data = pm.Data("age_data", age, dims="obs")
    child_data = pm.Data("child_data", child_codes, dims="obs")
    visit_data = pm.Data("visit_data", visit_idx, dims="obs")

    beta0 = pm.Normal("beta0", mu=0, sigma=5) # fixed effects priors
    beta1 = pm.Normal("beta1", mu=0, sigma=5)
    beta2 = pm.Normal("beta2", mu=0, sigma=5)

    sigma_u = pm.HalfNormal("sigma_u", sigma=1.0) # random effect
    u_raw = pm.Normal("u_raw", mu=0, sigma=1, dims="child")
    u = pm.Deterministic("u", sigma_u * u_raw, dims="child")

    sigma_v = pm.HalfNormal("sigma_v", sigma=1.0, dims="visit") # child-specific visit effects
    v_raw = pm.Normal("v_raw", mu=0, sigma=1, dims=("child", "visit"))
    v = pm.Deterministic("v", v_raw * sigma_v, dims=("child", "visit"))

    eta = (beta0 + beta1 * smoke_data + beta2 * age_data + u[child_data] + v[child_data, visit_data]) # linear predictor

    y_like = pm.Bernoulli("y_like", logit_p=eta, observed=y, dims="obs") # Bernoulli likelihood
```



Set Parameters of Sampling Process

```
idata = pm.sample(
   draws=2000,
   tune=2000,
   chains=4,
   cores=1,
   target_accept=0.95,
   random_seed=666,
   return_inferencedata=True
)

# posterior predictive
ppc = pm.sample_posterior_predictive(idata, random_seed=666)
idata.extend(pm.sample_posterior_predictive(idata, random_seed=666))
```



Diagnostics Plot

```
az.plot_trace(idata, var_names=["beta0", "beta1", "beta2", "sigma_u"])
plt.tight_layout()
plt.show()

az.plot_rank(idata, var_names=["beta0", "beta1", "beta2", "sigma_u"])
plt.tight_layout()
plt.show()

az.plot_ppc(idata, var_names=["y_like"])
plt.tight_layout()
plt.show()

az.plot_energy(idata)
plt.tight_layout()
plt.show()
```



Other Diagnostics

```
print("\nR-hat")
print(az.rhat(idata, var_names=["beta0", "beta1", "beta2", "sigma_u", "sigma_v"]))

print("\nESS")
print(az.ess(idata, var_names=["beta0", "beta1", "beta2", "sigma_u", "sigma_v"]))

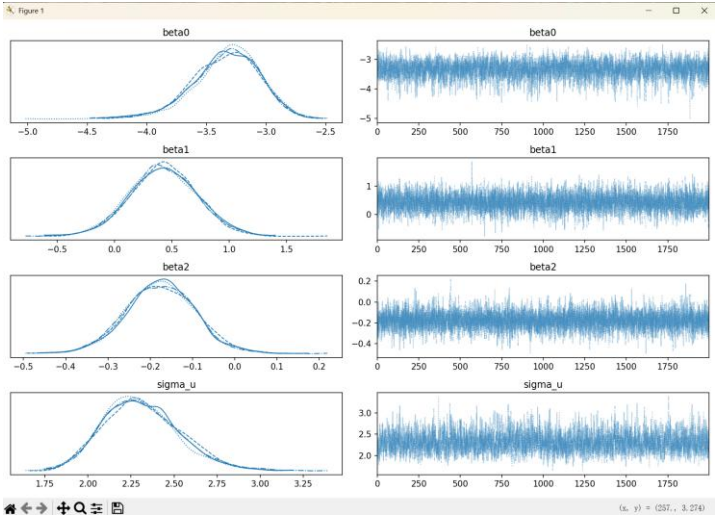
print("\nBFMI")
print(az.bfmi(idata))

n_div = int(idata.sample_stats["diverging"].sum().values)
print(f"\nNumber of divergences = {n_div}")
```

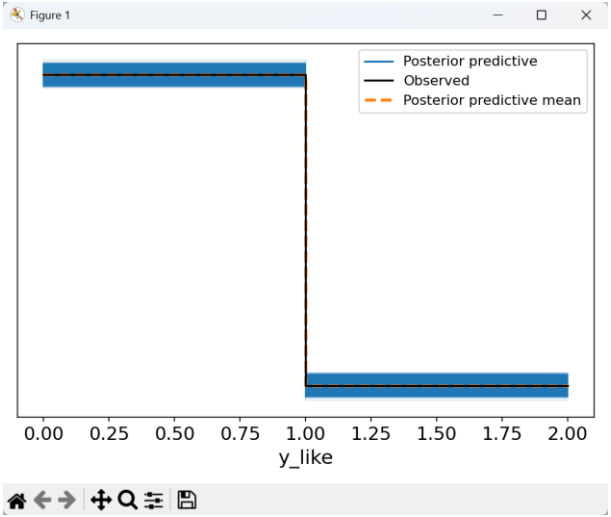
Two Real Examples

Example 2: Child's Wheeze Data

Trace Plot



PPC Plot



Interpretation of Diagnostics Plot

- **Trace Plot:** The four chains mix well and fluctuate around stable regions, with no obvious drift or chain separation, suggesting good convergence of the MCMC sampler. The posterior densities for all parameters are also highly consistent across chains.
- **PPC Plot:** The posterior predictive distribution closely overlaps with the observed distribution, indicating that the fitted model reproduces the binary wheeze outcomes reasonably well.

Estimation of Parameters

	mean	sd	hdi_2.5%	...	ess_bulk	ess_tail	r_hat
beta0	-3.335	0.283	-3.897	...	2291.152	2942.063	1.002
beta1	0.424	0.295	-0.130	...	7270.210	6757.081	1.001
beta2	-0.176	0.082	-0.350	...	5467.797	5480.628	1.000
sigma_u	2.311	0.223	1.888	...	1990.435	3368.775	1.003
sigma_v[0]	0.676	0.441	0.000	...	1390.375	2730.024	1.002
sigma_v[1]	0.669	0.425	0.001	...	1277.429	2597.398	1.001
sigma_v[2]	0.714	0.438	0.001	...	1258.918	2867.465	1.001
sigma_v[3]	0.449	0.329	0.000	...	2360.309	3871.283	1.002

The posterior means suggest a negative intercept ($\beta_0 = -3.335$), a positive smoking effect ($\beta_1 = 0.424$), and a modest negative age effect ($\beta_2 = -0.176$); the child-level random-effect variation is substantial ($\sigma_u = 2.311$), while the visit-specific random effects are smaller. The diagnostic statistics are excellent, with all $\widehat{R}$ values essentially equal to 1 and large effective sample sizes, indicating stable convergence and reliable posterior estimation.

The End



香港中文大學
The Chinese University of Hong Kong